

SPM: Lock-Free Programming in C++, concepts

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Atomic operations concept and simple example
- Memory Consistency Models
- C++ Memory Model
- Example implementations: ring buffer and bounded MPMC queue

C++ Atomics

- C++11 introduces *atomic types* that can be safely manipulated in a concurrent context without acquiring *locks* (i.e., `std::mutex`)
- Operations on atomic types are *indivisible* with respect to other threads
- In many scenarios, *atomic operations can be faster than using mutexes*, particularly under high concurrency when the critical section is very small
- C++ *atomics can be used to build non-blocking algorithms*, which avoid:
 - Thread suspension/resume overhead
 - **Deadlock**
 - **Priority inversion** issues: a thread with high priority is waiting to acquire a lock that is currently held by a low-priority thread

Atomic counting

- Let's illustrate the advantages of atomics with a simple example:
 - concurrent increment of a variable using both mutexes and atomics
 - Execution on the spmcluster front-end node

```
# elapsed time (mutex_multithreaded): 11.8701s
# elapsed time (atomic_multithreaded): 1.93948s
100000000 100000000
```

In this example, the use of atomics is approximately 6 times more efficient than the traditional approach using locks

Example code : *atomic_counting.cpp*

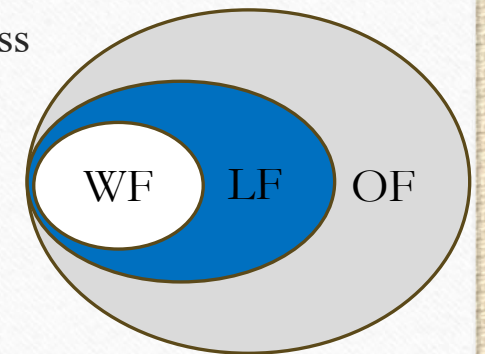
```
atomic_counting.cpp > ...
1  #include <iostream>
2  #include <cstdlib>
3  #include <vector>
4  #include <thread>
5  #include <atomic>
6  #include <mutex>
7  #include <hpc_helpers.hpp>
8
9  int main( ) {
10     std::mutex mutex;
11     std::vector<std::thread> threads;
12     const uint64_t num_threads = 10;
13     const uint64_t num_iters = 100'000'000;
14
15     auto lock_count =
16         [&] (uint64_t& counter, const auto& id) {
17             for (uint64_t i = id; i < num_iters; i += num_threads) {
18                 std::lock_guard<std::mutex> lock_guard(mutex);
19                 counter++;
20             }
21         };
22     auto atomic_count =
23         [&] (std::atomic<uint64_t>& counter, const auto& id) {
24             for (uint64_t i = id; i < num_iters; i += num_threads) {
25                 //counter++;
26                 //counter.fetch_add(1);
27                 std::atomic_fetch_add(&counter,1);
28             }
29         };
30     TIMERSTART(mutex_multithreaded)
31     uint64_t lock_counter = 0;
32     threads.clear();
33     for (uint64_t id = 0; id < num_threads; id++)
34         threads.emplace_back(lock_count, std::ref(lock_counter), id);
35     for (auto& thread : threads)
36         thread.join();
37     TIMERSTOP(mutex_multithreaded)
38
39     TIMERSTART(atomic_multithreaded)
40     std::atomic<uint64_t> atomic_counter(0);
41     threads.clear();
42     for (uint64_t id = 0; id < num_threads; id++)
43         threads.emplace_back(atomic_count, std::ref(atomic_counter), id);
44     for (auto& thread : threads)
45         thread.join();
46     TIMERSTOP(atomic_multithreaded)
47     std::cout << lock_counter << " " << atomic_counter << std::endl;
48 }
```

Challenges

- Lock-free programming avoids certain synchronization issues (e.g., deadlock, priority inversion) but introduces new challenges
- In general, lock-free programming is more complex and introduces problems related to **starvation** and **livelock**
 - **Starvation**: A thread is continually **denied the opportunity to make progress** while other threads continue to progress
 - **Livelock**: Threads continually interfere with each other in a way that **there is no real progress of the system**
- Careful design is necessary to avoid subtle race conditions, and to ensure both correctness and progress
- Understanding progress conditions is essential when designing lock-free data structures

Non-blocking progress guarantees

- Three main classes of progress guarantees (from strongest to weakest):
 - **Wait-freedom (WF)**: Every thread completes its operation in a bounded number of steps, independently of other threads
 - No starvation is possible for any thread
 - **Lock-freedom (LF)**: The system as a whole makes progress, i.e., at least one thread makes progress
 - Starvation is possible for some threads
 - Weaker than wait-freedom, it guarantees global progress but not fairness
 - **Obstruction-freedom (OF)**: A thread is guaranteed to complete only if it eventually runs in isolation for long enough
 - Under contention, livelock is possible
 - Weakest guarantee, it is meaningful only when contention is low or conflicts can be resolved



non-blocking algorithms

To build non-blocking algorithms in C++, the core tool is `std::atomic<T>` and its atomic operations

std::atomic<T>

- *std::atomic<T>* objects are neither copyable nor movable
- Main operations provided by std::atomic<T>
 - **load/store**: to read and write the atomic value
 - **exchange**: atomically replace the atomic value and return the old one
 - **compare exchange**: atomically replace the atomic value only if it matches an expected value
 - **fetch operations**: perform a read-modify-write in one atomic step
 - **wait/notify** (since C++20) provide blocking wait on atomic values similar in spirit to condition variables
- <https://en.cppreference.com/w/cpp/atomic/atomic>
- Example: in the *all_pair.cpp* code, we could implement the dynamic scheduling policy using an atomic counter instead of a mutex-protected variable

```
// while there are still rows to compute
while (lower < rows) {

    // update lower row with global lower row
    {
        std::lock_guard<std::mutex> lock_guard(mutex);
        lower = global_lower;
        global_lower += chunk_size;
    }

    ...
}
```

```
// while there are still rows to compute
while (lower < rows) {

    // update lower row with global lower row
    lower = std::atomic_fetch_add(&global_lower, chunk_size);

    ...
}
```


Atomic data types in C++

- C++ provides standard atomic types for **flags**, **booleans**, **pointers**, **integral types**, and **other trivially copyable types** via `std::atomic<T>`
- Available operations depend on the type
- If `std::atomic<myT> O` and `O.is_lock_free()` is false, then all operations (load, store, exchange, etc.) are guaranteed to be atomic, but the implementation may fall back to an internal lock-based implementation
 - The code you write remains correct even if the underlying HW does not natively support the corresponding data type

Example code : *query_atomics.cpp*

Operation	atomic_flag	atomic<bool>	atomic<T*>	atomic<integral-type>	atomic<other-type>
test_and_set	Y				
clear	Y				
is_lock_free		Y	Y	Y	Y
load		Y	Y	Y	Y
store		Y	Y	Y	Y
exchange		Y	Y	Y	Y
compare_exchange_weak, compare_exchange_strong		Y	Y	Y	Y
fetch_add, +=			Y	Y	
fetch_sub, -=			Y	Y	
fetch_or, =				Y	
fetch_and, &=				Y	
fetch_xor, ^=				Y	
++, --			Y	Y	

Compare-And-Swap (CAS)

- CAS (Compare-And-Swap) is a fundamental primitive to implement atomic assignments
- Two methods: *compare_exchange_strong* and *compare_exchange_weak*
 - *compare_exchange_weak* is more efficient but might suffer from spurious failures (it may return false even if the comparison yields true)
- Semantics:

```
atomic.compare_exchange_weak(T& expected, T desired);
```

 1. Compare the value *expected* with the value stored in the *atomic*
 2. If yes, then it sets the *atomic* to the value *desired*, otherwise it writes (overwrite!) the actual value stored in *atomic* into *expected*
 3. Return *true* if the swap operation in step 2 was successful, *false* otherwise
- CAS operations should always be performed in loops
 - CAS can fail due to races or spuriously when using *compare_exchange_weak*

Parallel Max-Reduction using CAS

- Compute the maximum of a sequence of 64-bit integers
- *false_max* (lines 14-19) occasionally computes the incorrect results since the condition tested in the if and the assignment (line 17) are two independent operations
 - Two or more threads might have read the same value before executing the assignment in a random order
- *correct_max*: uses a CAS in which the read and write operations are executed atomically

```
[torquati@spmln spmcode6]$ ./atomic_max
# elapsed time (incorrect_max): 2.0801s
# elapsed time (correct_max): 1.02373s
99999994 99999999
```

The incorrect version may be slower because many threads perform redundant atomic stores on the same cache line, causing higher contention and coherence traffic than the CAS-based version. This is not a general rule.

```
atomic_max.cpp > ...
1  #include <iostream>
2
3
4
5
6
7
8  int main( ) {
9      std::vector<std::thread> threads;
10     const uint64_t num_threads = 10;
11     const uint64_t num_iters = 100'000'000;
12
13     // WARNING: false_max produces incorrect results
14     auto false_max = [&] (std::atomic<uint64_t> &counter,
15                          const auto& id) -> void {
16         for (uint64_t i = id; i < num_iters; i += num_threads) {
17             if(i > counter) counter = i;
18         }
19     };
20     auto correct_max = [&] (std::atomic<uint64_t> &counter,
21                           const auto& id) -> void {
22         for (uint64_t i = id; i < num_iters; i += num_threads) {
23             auto previous = counter.load();
24             while (previous < i &&
25                   !counter.compare_exchange_weak(previous, i)) {}
26         }
27     };
28     TIMERSTART(incorrect_max);
29     std::atomic<uint64_t> false_counter(0);
30     threads.clear();
31     for (uint64_t id = 0; id < num_threads; id++)
32         threads.emplace_back(false_max, std::ref(false_counter), id);
33     for (auto& thread : threads)
34         thread.join();
35     TIMERSTOP(incorrect_max);
36
37     TIMERSTART(correct_max);
38     std::atomic<uint64_t> correct_counter(0);
39     threads.clear();
40     for (uint64_t id = 0; id < num_threads; id++)
41         threads.emplace_back(correct_max, std::ref(correct_counter), id);
42     for (auto& thread : threads)
43         thread.join();
44     TIMERSTOP(correct_max);
45     std::cout << false_counter << " " << correct_counter << std::endl;
46 }
```

Example code : *atomic_max.cpp*

Concepts

- The intuition says that a memory read should return the latest value written in a memory location
- In a single-threaded scenario this is straightforward, but what does «the latest value written» mean in a multiprocessor system executing multiple application threads?
- It depends on the **ordering** and **visibility** of memory operations, precisely the order in which memory operations performed by one thread become visible to other threads
- Modern CPUs and compilers can **reorder memory operations** in different ways for performance reasons. This affects when one thread's writes become visible to others
- A **Memory consistency model** defines the allowed behavior of reads and writes (i.e., loads and stores) to different addresses in a parallel system

Memory Consistency Models

- **Ordering & Visibility** of operations
 - Ordering rules define **when** and **how** changes in one thread become visible to others
 - They ensure predictable interactions among threads, even with compiler and/or hardware optimizations
- A **Memory Consistency Model** is a set of rules that defines how memory operations (reads and writes) behave and are observed across different threads or processors. It establishes the order in which operations appear to execute, defining when the result of a write operation by one thread becomes visible to other threads
 - **Ensure program correctness**. It prevents bugs in concurrent programs by clearly defining the order of operations
 - **Enable/Disable Optimization for performance**. Different models offer various options for introducing optimizations
- Atomics do not just guarantee atomic updates, they also enforce specific **memory ordering** and **visibility** rules

Why does memory ordering matter?

// initially, A = B = flag = 0

Thread1

```
Store A = 1;  
Store B = 1;  
Store flag = 1;
```

Thread2

```
while (Load flag==0); // spin  
valA = Load A;  
valB = Load B;  
print(valA, valB);
```

- We expect Thread2 should print A=B=1 (1,1). Without synchronization this is not guaranteed
- For some memory consistency models, the results (0,0), (1,0), and (0,1) are valid outputs
- Why? Because writes to different addresses (A, B, flag) can become visible at different times on different cores
- If the system includes caches, cache coherence only orders each location individually. Which result is printed depends on the memory consistency model (i.e., on the ordering of instructions)

Memory ordering is **not automatically** sequential

Cache Coherence vs. Memory Consistency

- **Memory consistency** defines correct program behavior
 - Defines the rules for the ordering and visibility of memory operations
 - It specifies how memory operations are **ordered** and observed across processors/cores
 - Consistency is part of the specification of architectures and is visible to system software
- **Cache coherence** concerns a single memory location replicated in multiple caches
 - It ensures that all processors see the same value for a given memory location, keeping the value consistent across caches
 - **Coherence is not visible at the software level**. We may see it as a side effect, e.g., false sharing, superlinear speedup, etc....
- Modern CPUs use coherence protocols to keep caches in sync. On top of that, memory consistency models define how compilers and hardware can reorder operations

Memory operation ordering

- **Program order**
 - A program defines a sequence of reads and writes
 - Within a single thread, operations appear in program order
- **Four types of memory operation ordering** ($A \rightarrow B$ denotes an ordering constraint):
 - $W_X \rightarrow R_Y$: write to X must be ordered before the subsequent read of Y
 - i.e., when a write comes before a read in **program order**, the memory model may require that ordering be preserved
 - $R_X \rightarrow R_Y$: read from X must be ordered before the subsequent read from Y
 - $R_X \rightarrow W_Y$: read from X must be ordered before the subsequent write to Y
 - $W_X \rightarrow W_Y$: write to X must be ordered before the subsequent write to Y

Different memory models may relax some or all of these ordering constraints

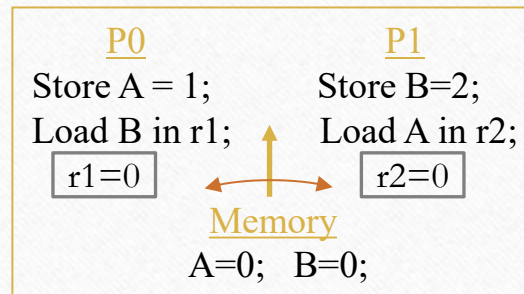
Sequential Consistency (SC)

- **Leslie Lamport (1979):**
 - “A multiprocessor is **sequentially consistent** if the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.”
 - First widely adopted formal definition of memory consistency. **SC is the most restrictive memory model**
 - Often described as the most intuitive model from a programmer’s perspective
 - Another equivalent definition for SC:
 - There exists a single **total order** of all loads and stores across all threads such that the value returned by each load is equal to the value of the most recent store to that location in this total order
 - There exists a global chronological order **of all memory operations** that is consistent with observed values
- **SC preserves all four memory operation orderings** ($W_X \rightarrow R_Y$, $R_X \rightarrow R_Y$, $R_X \rightarrow W_Y$, $W_X \rightarrow W_Y$)
 - Real hardware and compilers may reorder operations for efficiency, so additional instructions (**fences**) or stronger synchronization are often required to enforce SC in practice

Sequential Consistency (SC)

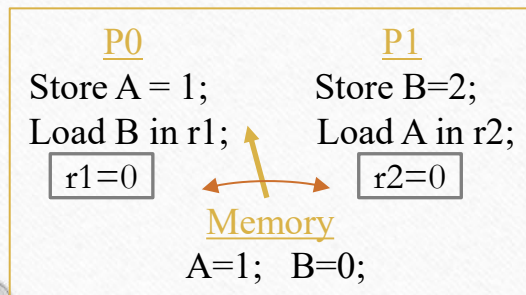
- Switching metaphor:** all processors issue loads/stores **in program order**, and the memory chooses any processor whose next operation is ready, executes that operation to completion, then chooses another processor

- Example:

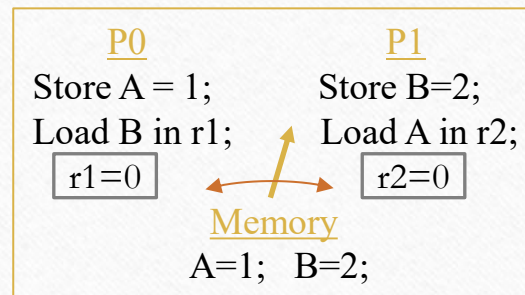


NOTE: the outcome $r1=0$ and $r2=0$ is not allowed by SC! SC allows the following ones:
 $r1 = 0, r2 = 1$
 $r1 = 2, r2 = 0$
 $r1 = 2, r2 = 1$

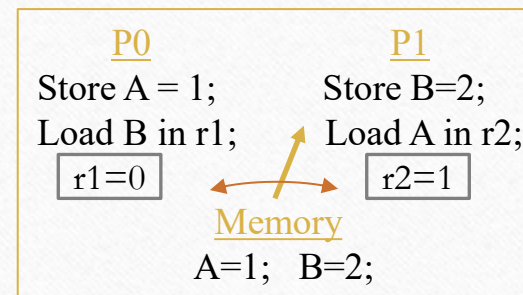
memory choose P0



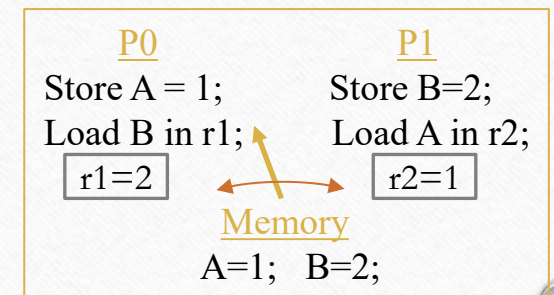
memory choose P1



memory choose P1

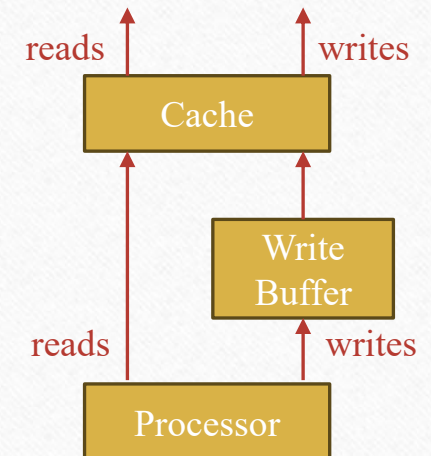


memory choose P0



Motivation for Relaxed Consistency

- **Relaxed memory consistency** models allow some program-order constraints to be relaxed
- Why? To improve performance by hiding memory latency
 - Overlapping memory access operations with other independent operations
- Modern processor uses **Write Buffers** to improve performance
 - A processor may allow a read to a different memory location to bypass older pending writes ($W_X \rightarrow R_Y$) to hide write latency
 - This is perfectly fine from the viewpoint of a single-thread control flow
 - This is a common optimization in modern processors



A **Write Buffer** is a small queue or memory area in the CPU that holds data from recent write operations that have not yet been fully committed to main memory (or the next level in the memory hierarchy). By storing these “pending” writes in a buffer, the processor can continue executing subsequent instructions without having to wait for each store to complete. If the CPU needs to read a location that’s in the write buffer, it can forward that data directly from the buffer instead of waiting for it to reach cache or main memory.

Since a read to a memory location can stall the processor until it is satisfied, loads may bypass older buffered stores to different locations

Total Store Order and Processor Consistency

- Total Store Order (TSO) and Processor Consistency (PC)
 - TSO relaxes $W_X \rightarrow R_Y$, while still preserving a single global order of stores
 - PC preserves the per-processor program order of stores, but does not require a single global order of stores from different processors

TSO: A load may bypass older pending stores in the local Write Buffer (a load cannot bypass an older store to the same location)

- All processors observe the same order for writes from any processor
- A load may bypass an earlier store to a different address, so a thread may temporarily observe an older value
- x86 uses a TSO-like memory consistency model. NOTE: TSO allows $r1=0$ and $r2=0$ in the previous example!

PC: All processors observe the writes issued by each processor in program order, but writes coming from different processors do not need to form a single global order

- It does not require a single global order for writes from different processors, especially when they affect different locations
- Requires explicit synchronization (e.g., memory fences) when coordinated updates across multiple locations are needed
- No modern architecture strictly implements PC. Many systems adopt either stronger or more relaxed models

Allowing reordering of writes

- **Partial Store Ordering (PSO)** also relaxes $W_X \rightarrow W_Y$
- Why? The processor might reorder write operations in the Write Buffer for performance reasons (e.g., one write might be a cache miss, while the other might be a cache hit that incurs lower management costs).
 - This is a valid optimization for a single-threaded program
 - For multithreaded applications, additional synchronization is needed
- **Why do architectures allow more aggressive reordering?**
 - Overlap multiple independent reads and writes in the memory system
 - Execute reads early and delay writes, further hiding memory latency
 - Out-of-order execution and speculative execution allow the CPU to keep its pipelines full
- One example of a fully relaxed model is **Weak Ordering**
 - ARM and POWER architectures adopt a very relaxed memory model for better performance, at the cost of increased programming complexity

Thread1 on P0	Thread2 on P1
A = 1;	while (flag==0) ; //spin
flag=1;	print A;

- Under PSO, P1 may observe the change to flag before the change to A, thus printing 0 (A initial value)
- Setting flag=1 and A=0 is not allowed by TSO, which preserves $W_X \rightarrow W_Y$

Enforce Ordering Constraints

- **Architectures provide primitives to enforce stricter memory ordering**
 - **Memory barrier instructions** (also called **fences**) restrict reordering, **but they are expensive**
 - A fence enforces ordering constraints between memory operations before and after the fence
 - There are different flavors of fences:
 - load fence (**RMB**): restricts reordering of loads
 - store fence (**WMB**): restricts reordering of stores
 - full fence (**MB**): restricts reordering of both loads and stores
- Other synchronization primitives may also enforce ordering, often on specific memory locations
 - E.g., **test_and_set**, **compare-and-swap**, and all **Read-Modify-Write (RMW)** operations
- **Programmers, and in some cases the compiler/runtime, must properly insert synchronizations to ensure correct ordering**
 - Locks and other synchronization operations may implicitly provide fence-like ordering guarantees

program:

```
reads and writes  
reorderable  
...  
FENCE  
...  
reads and writes  
reorderable  
...  
FENCE
```

Thread1
on P0

```
A = 1;  
WMB;  
flag=1;
```

Thread2
on P1

```
while (flag==0) ; //spin  
RMB;  
print A;
```

Using fences, we can restore the intended ordering in this example under a relaxed memory consistency model

Data-Race-Free (DRF) guarantee

- **Theorem:** If a program is DRF, then its observable behavior is consistent with some Sequentially Consistent execution
 - All operations behave as if they occur in a **single global order** that respects each thread's program order
- Instruction reordering does not break correctness if the program is DRF
 - Programs that use synchronization correctly (mutexes, barriers, atomics, etc.) to order all conflicting accesses are DRF
- **Sequential Consistency does not eliminate data races by itself.** Programmers must explicitly ensure data-race freedom through synchronization

The rationale behind memory reordering is the principle “**Optimize for the common case**”. Since most memory accesses do not conflict, architectures avoid incurring the overhead of strict ordering for every operation.

Data Race: two or more threads concurrently access the same memory location, at least one executes a write, and there is no synchronization between accesses

Programming languages need Memory Model

- **The compiler may reorder instructions for performance reasons independently from the low-level micro architectures**
 - It is generally accepted that a compiler can reorder ordinary reads from and writes to memory almost arbitrarily, provided the reordering cannot change the observed **single-threaded** execution of the code
- **Modern C++** (starting from C++11) defines a memory model that **guarantees SC for DRF programs**
 - **The compiler maps language-level atomics and fences onto the underlying HW memory model**
 - No guarantees if the program contains data races
 - Unsynchronized programs contain data races, i.e., the output of the program depends on the relative speed of processors executing threads (**non-deterministic program results**)
- C++11 introduced, at the language level, atomic operations and fences to specify how memory ordering works for both atomic and non atomic operations
- This helps programmers control visibility and ordering of memory accesses across threads, making the program portable across different micro architectures

C++ memory model basics

- From this point on, we move from HW memory models to the C++ language-level memory model
- C++ provides the user with six memory ordering options for atomic types:
 - 1. `std::memory_order_relaxed`; 2. `std::memory_order_consume`; 3. `std::memory_order_seq_cst`;
4. `std::memory_order_acquire`; 5. `std::memory_order_release`; 6. `std::memory_order_acq_rel`
- *Load*, *Store*, and *Read-Modify-Write* (RMW) operations accept **explicit** ordering values
 - The **default** memory ordering is the strictest one: **sequential consistency** (`std::memory_order_seq_cst`)
- *Store* operations can be *relaxed*, *release*, or *seq_cst*
- *Load* operations can be *relaxed*, (*consume* **), *acquire*, or *seq_cst*
- *RMW* operations can be *relaxed*, (*consume* **), *acquire*, *release*, *acq_rel* or *seq_cst*
- Each ordering imposes different constraints on how operations can be reordered around that atomic operation

** `std::memory_order_consume`, is unimplemented by most compilers
the C++ standard advises against using it because of the difficulty in implementing it correctly and consistently

Happens-before relation (HB)

- **Happens-before (HB):** If A happens-before B, then the side effects of A are visible to B
 - if A *HB* B, then:
 - Every write/effect performed by A is **visible** to B
 - B cannot observe a state that ignores A's effects (B cannot be **reordered** before A)
- HB is a transitive partial order (if A *HB* B and B *HB* C, then A *HB* C)
- How HB can be established
 - **Sequenced-before (intra-thread):** within one thread, actions are ordered according to the C++ execution rules
 - **Synchronizes-with (inter-thread):**
 - `unlock(m)` *synchronizes-with* `lock(m)` on the same mutex m
 - `notify_one/all()`, together with the associated mutex, *synchronizes-with* the successful return of `wait()` after the waiting thread has re-acquired the same mutex
 - the completion of a thread *synchronizes-with* the successful return from `join()`
 - a store-release on an atomic variable *synchronizes-with* a load-acquire that reads that value
- Two conflicting non-atomic accesses to the same variable, at least one of which is a write, and without *HB* relation, cause a **data race** (in C++ called Undefined Behavior)

We need the HB relation to reason about program correctness when using atomics

C++ memory ordering

- Types of memory ordering (https://en.cppreference.com/w/cpp/atomic/memory_order)
 - **Sequential consistency**: all seq_cst atomic operations across all threads appear in one single global order that is consistent with each thread's program order
 - **Acquire** (load and RMW operations) : it prevents all subsequent loads/stores in the current thread from being moved to a position before the current atomic load operation. If the acquire reads a value produced by a matching release on the same atomic object, it synchronizes-with that release (establishes HB)
 - **Release** (store and RMW operations): it prevents all prior loads/stores in the current thread from being moved to a position after the current atomic store operation. Pairs with a later acquire load on the same atomic object.
 - **Relaxed** : no restrictions in memory reordering of surrounding load/store operations. Only the atomicity of the actual operation is guaranteed (no HB relation)
 - **Acquire-Release** (RMW operations): it is a combination of the Acquire and Release modes, simultaneously loads the current value in Acquire mode and stores a new value in the atomic variable in Release mode. It ensures no subsequent operations can move before it, and no prior operations can move after it.

RMW operations: exchange, compare-exchange, test_and_set on atomic_flag, and fetch_*

SC implies a total ordering

- The SC semantics requires *a single total ordering* over all atomic operations with ordering `std::memory_order_seq_cst`
 - Non-atomic accesses are not part of that ordering
- The assert at line 37 can never fire, because either the store to x (line 9) or the store to y (line 12) *must happen before* at least one of the loads on x or y (we don't know the order)
- If the load at line 16 returns false, it means that the store to x at line 9 *must happen before* the store to y at line 12 and thus the load at line 21 must return true
 - The other way around is also valid, if the load at line 21 returns false, the load at line 16 must return true
- SC is the most expensive memory ordering, it constrains both compiler and hardware more than weaker orderings

```
1  #include <atomic>
2  #include <thread>
3  #include <assert.h>
4
5  std::atomic<bool> x,y;
6  std::atomic<int> z;
7
8  void write_x() {
9      x.store(true,std::memory_order_seq_cst);
10 }
11 void write_y() {
12     y.store(true,std::memory_order_seq_cst);
13 }
14 void read_x_then_y() {
15     while(!x.load(std::memory_order_seq_cst));
16     if(y.load(std::memory_order_seq_cst))
17         ++z;
18 }
19 void read_y_then_x() {
20     while(!y.load(std::memory_order_seq_cst));
21     if(x.load(std::memory_order_seq_cst))
22         ++z;
23 }
24
25 int main() {
26     x=false;
27     y=false;
28     z=0;
29     std::thread a(write_x);
30     std::thread b(write_y);
31     std::thread c(read_x_then_y);
32     std::thread d(read_y_then_x);
33     a.join();
34     b.join();
35     c.join();
36     d.join();
37     assert(z.load()!=0); // it can never fire!
38 }
```

Relaxed Ordering

- Under `std::memory_order_relaxed`, each atomic operation is guaranteed to be atomic, but no additional ordering constraints are imposed
 - With relaxed ordering there is no happens-before among atomic operations. Operations on different atomics can be observed in any order
- In this new code, the assert at line 27 *can* fire
 - Here we use a simpler 2-thread example instead of 4 threads, but the conceptual problem is the same: relaxed ordering does not enforce any ordering on different atomics
 - The load at line 15 can return false even if the load at line 14 returns true
 - This means one thread could see the store to y happen before the store to x, while the other thread sees the store to x but not y

```
1  #include <atomic>
2  #include <thread>
3  #include <assert.h>
4
5  std::atomic<bool> x,y;
6  std::atomic<int> z;
7
8  void write_x_then_y() {
9      x.store(true,std::memory_order_relaxed);
10     y.store(true,std::memory_order_relaxed);
11 }
12
13 void read_y_then_x() {
14     while(!y.load(std::memory_order_relaxed));
15     if(x.load(std::memory_order_relaxed))
16         ++z;
17 }
18
19 int main() {
20     x=false;
21     y=false;
22     z=0;
23     std::thread a(write_x_then_y);
24     std::thread b(read_y_then_x);
25     a.join();
26     b.join();
27     assert(z.load()!=0); // it can fire!
28 }
```

NOTE: on some CPUs (e.g., x86) you may not be able to see the assert firing due to the store ordering in HW

Example code : *relaxed_ordering.cpp*

Acquire and Release Ordering

- With acquire-release ordering the synchronization is **pairwise** between the thread that does the **release** and the thread that does the **acquire**. Different threads can see different ordering
 - A release operation synchronizes-with an acquire operation that reads the value written.
- In this new code, the assert at line 37 *can* fire because the load at line 15 synchronizes only with the store at line 9, and the load at line 20 synchronizes only with the store at line 12. The ordering from the release to the acquire in each case has no effect on the operations in the other threads

NOTE: failure is possible on weak models (ARM/RISC-V/POWER)
On x86 it does not fail.

```
1  #include <atomic>
2  #include <thread>
3  #include <assert.h>
4
5  std::atomic<bool> x,y;
6  std::atomic<int> z;
7
8  void write_x() {
9      x.store(true,std::memory_order_release);
10 }
11 void write_y() {
12     y.store(true,std::memory_order_release);
13 }
14 void read_x_then_y() {
15     while(!x.load(std::memory_order_acquire));
16     if(y.load(std::memory_order_acquire))
17         ++z;
18 }
19 void read_y_then_x() {
20     while(!y.load(std::memory_order_acquire));
21     if(x.load(std::memory_order_acquire))
22         ++z;
23 }
24
25 int main() {
26     x=false;
27     y=false;
28     z=0;
29     std::thread a(write_x);
30     std::thread b(write_y);
31     std::thread c(read_x_then_y);
32     std::thread d(read_y_then_x);
33     a.join();
34     b.join();
35     c.join();
36     d.join();
37     assert(z.load()!=0); // it can fire!
38 }
```

Example code : *acq-rel_ordering.cpp*

From global ordering to pairwise synchronization

- In the previous examples, we used the same 4-thread pattern to compare three memory-ordering models:
 - `seq_cst` gives a single global order over all `seq_cst` atomic operations
 - `relaxed` guarantees atomicity only, with no additional inter-thread ordering
 - `acquire/release` gives pairwise synchronization only, not a global order across different atomics
- `seq_cst` is strong enough to prevent the assertion failure, `acquire/release` is not enough, because the example requires a global ordering property
- We now switch to a different pattern where a thread publishes data and then signals that the data is ready
- In this pattern, pairwise synchronization can replace `seq_cst` with lower cost

Relaxing SC with the Acquire and Release Ordering

- The load from y, at line 13 *synchronizes-with* the store at line 10. The store to x at line 9 is *sequenced-before* the store to y (i.e., such store cannot be moved after the store to y) and thus *happens-before* the load from y at line 13, which is *sequenced-before* the load from x at line 14 (the load from x cannot be moved before the load from y).
 - To establish synchronization, the acquire must read from the corresponding release. In this example, this is why the load at line 13 must be in a loop

HB chain:

x.store(relaxed) sequenced-before y.store(release) →

y.store(release) synchronizes-with y.load(acquire) →

y.load(acquire) sequenced-before x.load(relaxed)

By transitivity, x.store happens-before x.load, so x.load must see true

```
1  #include <atomic>
2  #include <thread>
3  #include <assert.h>
4
5  std::atomic<bool> x,y;
6  std::atomic<int> z;
7
8  void write_x_then_y() {
9      x.store(true,std::memory_order_relaxed);
10     y.store(true,std::memory_order_release);
11 }
12 void read_y_then_x() {
13     while(!y.load(std::memory_order_acquire));
14     if(x.load(std::memory_order_relaxed))
15         ++z;
16 }
17 int main() {
18     x=false;
19     y=false;
20     z=0;
21     std::thread a(write_x_then_y);
22     std::thread b(read_y_then_x);
23     a.join();
24     b.join();
25     assert(z.load()!=0); // it can never fire!
26 }
```

C++ Fences

- Fences are operations that impose ordering constraints **without modifying a memory location**
 - Fences are often used to add ordering constraints around **relaxed** atomic operations.
- C++11 provides `std::atomic_thread_fence(memory order)`
 - Release fence**: prevents prior loads/stores to move after the fence
 - Acquire fence**: prevents subsequent loads/stores to move before the fence
 - Acq_rel fence**: combines the previous two, acting as a full barrier
 - Seq_cst fence**: acts as a full barrier and participates in the global SC order for seq_cst operations
- Fences **with** atomic operations can also be used to order non-atomic operations, like in the code snippet here
 - The store at line 10 **happens-before** the load at line 17. The store at line 10 cannot move past the release fence, the load at line 17 cannot move before the acquire fence, and the relaxed store and load on y link the two threads

```
1  #include <atomic>
2  #include <thread>
3  #include <assert.h>
4
5  bool x=false;
6  std::atomic<bool> y;
7  std::atomic<int> z;
8
9  void write_x_then_y() {
10     x=true;
11     std::atomic_thread_fence(std::memory_order_release);
12     y.store(true,std::memory_order_relaxed);
13 }
14 void read_y_then_x() {
15     while(!y.load(std::memory_order_relaxed));
16     std::atomic_thread_fence(std::memory_order_acquire);
17     if(x)
18         ++z;
19 }
20 int main()
21 {
22     x=false;
23     y=false;
24     z=0;
25     std::thread a(write_x_then_y);
26     std::thread b(read_y_then_x);
27     a.join();
28     b.join();
29     assert(z.load()!=0); // it can never fire!
30 }
```


All-pairs distance Matrix revised

- Dynamic distribution with chunk size = 1: replacing the mutex-protected chunk counter with an atomic *fetch_add*

```
// atomic global variable
std::atomic<index_t> global_lower{0};

auto dynamic_block_cyclic = [&] (const index_t& id) -> void {
    // assume we have not done anything
    index_t lower = 0;
    // while there are still rows to compute
    while (lower < rows) {

        lower = global_lower.fetch_add(chunk_size, std::memory_order_relaxed);

        ...
    }
}
```

Here `std::memory_order_relaxed` is sufficient because we need only atomicity

Dynamic distribution c=1 (sequential 704s)		
#threads	std::mutex	std::atomic
20	59.32	59.36
40	34.22	34.10
60	65.21	61.07

Results obtained running the *all_pair.cpp* implementation for the MNIST dataset on the cluster front-end node

In this example, atomics do not help much at low contention, but they are better as contention increases

Example code : *all_pair-atomic.cpp*

Spin-lock

- A spin lock is still a lock: only one thread makes progress in the critical section
- It can be implemented with an *atomic_flag*
- *std::atomic_flag* is guaranteed to be lock-free on all platforms
 - Core operations: *test_and_set* and *clear*
 - C++20 also adds *test*, *wait*, *notify_one/all*
- *lock()* first tries *test_and_set(acquire)*. If it fails, it polls with *test(relaxed)* until the flag changes, then retries
- *unlock(release)* **synchronizes-with** *lock(acquire)* on the same atomic objects
 - release on unlock publishes the writes done inside the critical section
 - acquire on lock makes those writes visible to the next lock owner

```
3  #include <vector>
4  #include <atomic>
5
6  class spinLock {
7      std::atomic_flag flag;
8  public:
9      spinLock(): flag(ATOMIC_FLAG_INIT) {}
10     void lock() {
11         while(flag.test_and_set(std::memory_order_acquire)) {
12             while(flag.test(std::memory_order_relaxed)) ; // spin
13         }
14     }
15     void unlock() {
16         flag.clear(std::memory_order_release);
17     }
18 };
19
20 int main() {
21     const int nthreads=40;
22     std::vector<size_t> results(nthreads);
23     std::atomic<int64_t> counter{1000000};
24     std::vector<std::thread> v;
25     spinLock SP;
26
27     auto F = [&](int id) {
28         int success{0};
29         while(true) {
30             SP.lock();
31             if (counter.load() > 0) {
32                 ++success;
33                 --counter;
34             } else { SP.unlock(); break;}
35             SP.unlock();
36         }
37         results[id] = success;
38     };
39
40     for (int i=0; i<nthreads; ++i)
41         v.emplace_back(F, i);
42
43     for (auto& t : v) t.join();
44 }
```

Example code : *spin-lock.cpp*

Max-Reduction revised

- Code *atomic_max_v2.cpp*. The computation of the sequence max implemented with: `std::mutex`, `spinLock`, and atomic CAS

```
g++ -std=c++20 -I. -I./include -Wall -O3 -march=native -o atomic_max
[torquati@spmln spmcode4]$ for i in {1..5}; do ./atomic_max_v2; done
# elapsed time (max_with_lock): 11.3537s
# elapsed time (max_with_spinlock): 20.7907s
# elapsed time (max_with_atomic): 1.35059s
99999999 99999999 99999999
# elapsed time (max_with_lock): 11.9597s
# elapsed time (max_with_spinlock): 5.58074s
# elapsed time (max_with_atomic): 0.834882s
99999999 99999999 99999999
# elapsed time (max_with_lock): 11.1232s
# elapsed time (max_with_spinlock): 10.7354s
# elapsed time (max_with_atomic): 0.792781s
99999999 99999999 99999999
# elapsed time (max_with_lock): 11.1827s
# elapsed time (max_with_spinlock): 19.8105s
# elapsed time (max_with_atomic): 1.00384s
99999999 99999999 99999999
# elapsed time (max_with_lock): 11.0098s
# elapsed time (max_with_spinlock): 16.1718s
# elapsed time (max_with_atomic): 1.0822s
99999999 99999999 99999999
[torquati@spmln spmcode4]$
```

Execution time over 5 runs, min - max (s)

std::mutex	spinLock	std::atomic
11.0 – 11.9	5.6 – 20.7	0.8 – 1.3

- Atomic-based version is consistently faster and reasonably stable
- `spinLock`-based implementation (almost) always worse than mutex lock. Under contention, spin locks waste CPU cycles and increase coherence traffic, so they can easily perform worse than mutexes
 - Implementing efficient `spinLock` is hard

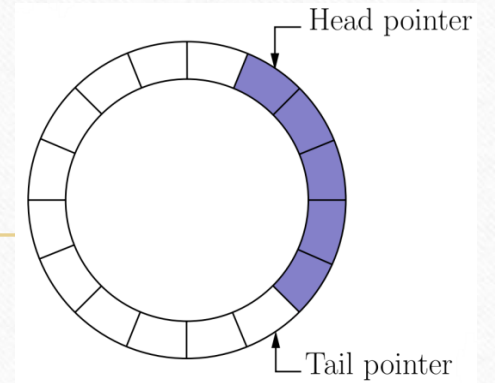
Example code : *atomic_max_v2.cpp*

std::barrier and a custom atomic barrier

- The `std::barrier` is a synchronization primitive introduced in C++20 that allows a fixed number of threads to wait until all have reached a certain point in the code
- The barrier is reusable, so it can be used in iterative or phased algorithms where you need multiple barrier points across iterations
- When you expect that the waiting time at the barrier to be very short, a busy-waiting implementation (spin) may have less overhead than a blocking barrier because it avoids the overhead of context switches (provided that the machine is dedicated and the threads are not oversubscribed)
 - Blocking barriers usually put threads to sleep (using OS-level synchronization)
- If the wait times can be longer or are unpredictable, using a blocking barrier (such as `std::barrier`) is generally preferable because it lets the OS suspend waiting threads, freeing up CPU resources for other work

Example code showing how to use a `std::barrier` and a possible implementation of the `spin-barrier` : `spinbarrier-wait.cpp`

SPSC ring buffer



- Bounded FIFO queue implemented as a fixed-size circular array
- SPSC (Single Producer / Single Consumer) pattern. Lock-free implementation: no mutex; only two indices (**head**, **tail**), updated by different threads. Tail is updated only by the producer and head only the consumer
- The implementation proposed is cache-friendly, with no dynamic allocations: pre-allocated array; head and tail are placed on different cache lines to avoid false sharing
 - `alignas(std::hardware_destructive_interference_size)` or `alignas(64)`
- Typical use: **producer–consumer pipelines and stage-to-stage communications**

SPSC ring buffer, synchronization

- head and tail are `std::atomic<std::size_t>`. Each index has a single writer. No CAS is needed.
- Publish/consume rule (release/acquire):
 - Producer: write payload into `buffer[tail]`
 - Producer: publish it with `tail.store(next, std::memory_order_release)`
 - Consumer: observe tail with `tail.load(std::memory_order_acquire)` before reading payload
 - Symmetrical with the head for the consumer advancing and the producer observing
- The release on tail/head `synchronize-with` the acquire on tail/head, so payload writes happens-before payload read (no explicit fence needed)
- Accesses to the index owned by the current thread can often use `memory_order_relaxed`
 - Acquire is needed when observing the other side's index
 - Release is needed when advancing your own index after producing or consuming the payload

SPSC ring buffer, progress

- `try_push` and `try_pop` are **wait-free**: each operation completes in a bounded number of steps, with no retry loops or blocking
 - This is possible because each index has a single writer
 - If the buffer is full or empty, the non-blocking operation fails fast and returns immediately
- A blocking API can be built on top of the same SPSC buffer: when `try_push` or `try_pop` fails, the caller waits until the state changes, then retries
- In C++20, this can be implemented efficiently with **`atomic::wait`** / **`notify_one`**, avoiding busy-waiting when the queue remains full or empty
- Benchmarks show that busy-wait is best when waiting times are very short, while blocking variants become competitive when the consumer is slower
- This simplicity does not extend to MPMC queues, where multiple threads must coordinate on shared logical positions

Example code contained in the *ringBuffer* directory

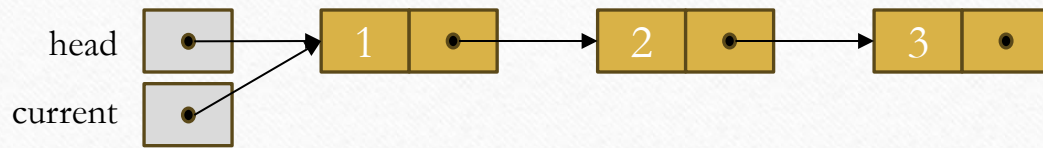
The ABA problem

- Pointer-based lock-free data structures implemented with CAS, **ABA** is a classic correctness problem
- The ABA problem occurs when a location's value changes from *A* to *B* and then back to *A*. A later reads of *A* cannot detect that an intermediate change happened
- **Typical ABA scenario**
 - Thread T1 reads value A
 - T1 is suspended
 - Another thread changes $A \rightarrow B \rightarrow A$
 - T1 resumes, CAS still sees A, but the logical state is no longer the same

Detailed steps leading to the **ABA** problem

- 1 Thread 1 reads an atomic variable, *x*, and finds it has value A.
- 2 Thread 1 performs some operation based on this value, such as dereferencing it (if it's a pointer) or doing a lookup, or something.
- 3 Thread 1 is stalled by the operating system.
- 4 Another thread performs some operations on *x* that change its value to B.
- 5 A thread then changes the data associated with the value A such that the value held by thread 1 is no longer valid. This may be as drastic as freeing the pointed-to memory or changing an associated value.
- 6 A thread then changes *x* back to A based on this new data. If this is a pointer, it may be a new object that happens to share the same address as the old one.
- 7 Thread 1 resumes and performs a compare/exchange on *x*, comparing against A. The compare/exchange succeeds (because the value is indeed A), but this is the wrong A value. The data originally read at step 2 is no longer valid, but thread 1 has no way of telling and will corrupt the data structure.

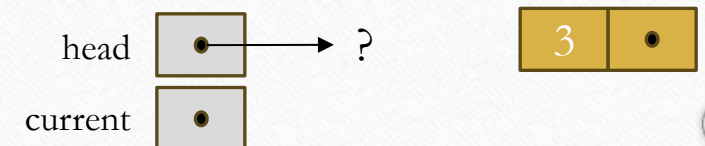
ABA Example



// typical pseudocode of a pop operation of a pointer-based lock-free stack

```
Node* pop() {  
    Node* current = head;  
    while (current) {  
        if (head == current &&  
            CAS(&head, current, current->next)) break;  
        current = head;  
    }  
    return current;  
}
```

1. Thread 1 tries to pop the head node, but it is suspended just before executing the CAS (just after reading `current->next`)
2. Thread 2 starts and pops the head node (1) and again the head node (2)
3. Thread 2 pushes the new node (containing 4),
reusing the node that contained the initial head node (i.e., 1).
This can happen because the memory allocator recycled the address
4. Thread 1 resumes, and the CAS succeeds because the *current* pointer matches (raw pointers look identical even though the node content has changed), thus removing the actual head node (4) and setting the head to a node that is no longer in the list, potentially deleted. The stack is corrupted!



Avoiding ABA

- The common approach to avoid the ABA problem is to add an extra **tag (version number)** to the data
 - The tag is incremented every time the pointer is updated successfully
 - The CAS operation works on both the pointer and the tag as a single atomic operation
 - This way the CAS will fail even if the address is the same
 - If the pointer was changed twice ($A \rightarrow B \rightarrow A$), the tag would have changed
 - It may require CAS2 (double-word CAS, not available on all architectures)
- Another option is to implement **safe memory reclamation**
 - The idea is to prevent premature reuse of nodes that may still be observed by other threads
 - **Hazard pointers** are a common example of safe memory reclamation

Hazard pointers address the problem of safely freeing or reusing nodes that might still be accessed by other threads.

- Each thread before reading or modifying a shared node, “publicly announce” it is using that node using a per-thread hazard pointer list
- If a pointer appears in any hazard pointer list, it cannot be reclaimed
- Once the thread is done with the node, it removes the pointer from the list. If no other thread has that pointer in its hazard list the node can be safely freed.

ABA is not only a CAS issue, it is also a memory-reuse issue

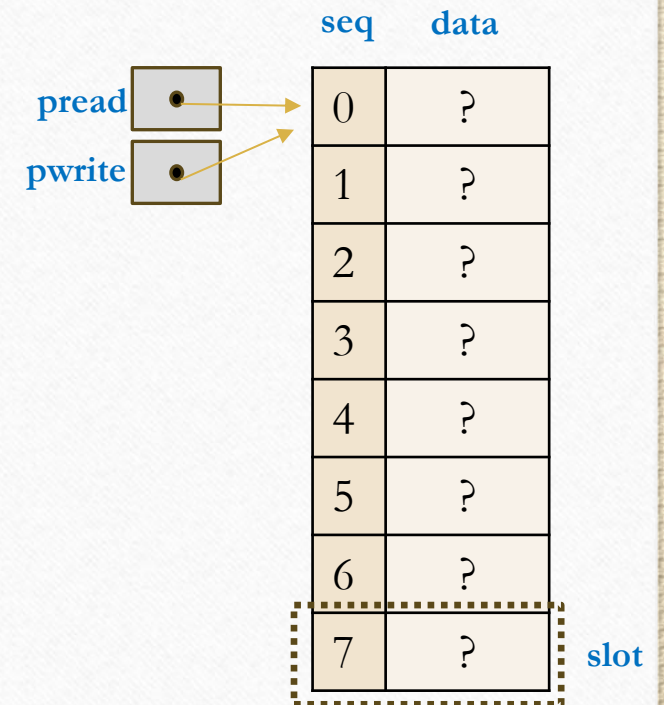
Moving to MPMC queue

- ABA is a classic issue in pointer-based lock-free linked structures
- The bounded MPMC queue we study in the next slide is different:
 - it is array-based, not list-based
 - it uses a fixed-size ring buffer
 - it does not rely on CAS over a shared head pointer of a linked list
- Its per-slot **sequence number** is used to track slot state across ring wrap-around, not to solve linked-structure ABA
- Therefore the **sequence number** is used to implement a slot-state protocol and it is not an ABA tag

Bounded MPMC queue

1/2

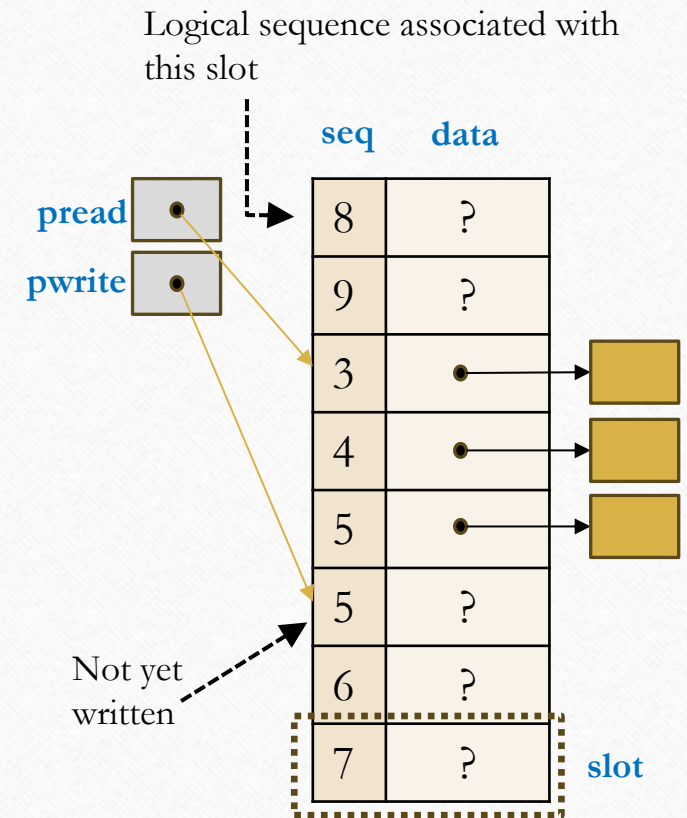
- Use case: a bounded lock-free MPMC FIFO queue based on CAS-loop for the enqueue (push) and dequeue (pop) operations
- The queue is a fixed-size ring-buffer with two global positions (**pread** and **pwrite**)
- Each slot stores a payload (pointer in our implementation) and a sequence number **seq**
 - **seq** encodes the logical state of that slot
 - it tells whether the slot is free for the current producer position, full for the current consumer position, or belongs to a later wrap-around of the ring
- The same physical slot is reused across ring cycles, and **seq** tells which logical cycle it belongs to



Bounded MPMC queue

2/2

- **push**: If the sequence number is equal to the tail (pwrite), then try to book the slot with a CAS operation atomically updating the tail pointer. If it succeeds the sequence is updated after storing the data
 - if $\text{slot.seq} == \text{pwrite}$, the slot is free for this enqueue position
 - try to reserve that position by CAS on pwrite and write the data if the CAS succeeds
 - publish the already written slot with $\text{seq} = \text{pwrite} + 1$
- **pop**: If the next head ($\text{pread} + 1$) is equal to the sequence number, then try to reserve the slot with a CAS operation atomically updating the head. If it succeeds the sequence is updated after reading the data to the next sequence value
 - if $\text{slot.seq} == \text{pread} + 1$, the slot contains valid data for this dequeue position
 - try to reserve that position by CAS on pread and read the data if the CAS succeeds
 - mark the slot reusable for the next wrap-around with $\text{seq} = \text{pread} + \text{capacity}$



Example code : *bounded_mpmc.cpp*

Lock-free vs. Lock-based synchronization

- Lock-free (*non-blocking*) algorithms guarantee system-wide progress: even if one thread stalls, some thread can still make progress (**lock-free progress property**).
 - Blocking algorithms can lead to deadlocks, convoying, or priority inversion
- Beyond simple cases, lock-free concurrency is significantly more complex than blocking lock-based concurrency
 - Memory operation ordering matters
 - The **ABA problem** and the associated **safe memory reclamation** technique are important aspects to consider in pointer-based data structures
- The relative performance of blocking vs. non-blocking algorithms depends significantly on the amount of concurrency (and the number of cores). It also depends on the specific algorithm used.
 - On dedicated environments with high concurrency and thus high contention on shared variables, carefully designed **lock-free algorithms are generally more scalable**

Suggested Readings

- Chapter 5 of the «Parallel Programming Concept and Practice» book
- «C++ Concurrency in Action» second edition by Anthony Williams
 - <https://www.manning.com/books/c-plus-plus-concurrency-in-action-second-edition>
 - Chapter 5 provided as additional material
- «A Primer on Memory Consistency and Cache Coherence» by Daniel J. Sorin, Mark D. Hill, and David A. Wood (A very detailed document on all aspects of Memory Consistency Models)

Some links: `std::memory_order` https://en.cppreference.com/w/cpp/atomic/memory_order
 `std::atomic_thread_fence` https://en.cppreference.com/w/cpp/atomic/atomic_thread_fence

Research papers:

- «Lock-free locks revised» <https://dl.acm.org/doi/10.1145/3503221.3508433>
- «The State-of-the-Art LCRQ Concurrent Queue Algorithm Does NOT Require CAS2»
<https://dl.acm.org/doi/10.1145/3572848.3577485>